
2.4 Exceptions

Exceptions are unexpected events that occur during the execution of a program. An exception might result due to an unavailable resource, unexpected input from a user, or simply a logical error on the part of the programmer. In Java, exceptions are objects that can be *thrown* by code that encounters an unexpected situation, or by the Java Virtual Machine, for example, if running out of memory. An exception may also be *caught* by a surrounding block of code that “handles” the problem in an appropriate fashion. If uncaught, an exception causes the virtual machine to stop executing the program and to report an appropriate message to the console. In this section, we discuss common exception types in Java, as well as the syntax for throwing and catch exceptions within user-defined blocks of code.

2.4.1 Catching Exceptions

If an exception occurs and is not handled, then the Java runtime system will terminate the program after printing an appropriate message together with a trace of the runtime stack. The stack trace shows the series of nested method calls that were active at the time the exception occurred, as in the following example:

```
Exception in thread "main" java.lang.NullPointerException
  at java.util.ArrayList.toArray(ArrayList.java:358)
  at net.datastructures.HashChainMap.bucketGet(HashChainMap.java:35)
  at net.datastructures.AbstractHashMap.get(AbstractHashMap.java:62)
  at dsaj.design.Demonstration.main(Demonstration.java:12)
```

However, before a program is terminated, each method on the stack trace has an opportunity to *catch* the exception. Starting with the most deeply nested method in which the exception occurs, each method may either catch the exception, or allow it to pass through to the method that called it. For example, in the above stack trace, the ArrayList.java method had the first opportunity to catch the exception. Since it did not do so, the exception was passed upward to the HashChainMap.bucketGet method, which in turn ignored the exception, causing it to pass further upward to the AbstractHashMap.get method. The final opportunity to catch the exception was in the Demonstration.main method, but since it did not do so, the program terminated with the above diagnostic message.

The general methodology for handling exceptions is a *try-catch* construct in which a guarded fragment of code that might throw an exception is executed. If it throws an exception, then that exception is *caught* by having the flow of control jump to a predefined **catch** block that contains the code to analyze the exception and apply an appropriate resolution. If no exception occurs in the guarded code, all catch blocks are ignored.

A typical syntax for a **try-catch statement** in Java is as follows:

```
try {  
    guardedBody  
} catch (exceptionType1 variable1) {  
    remedyBody1  
} catch (exceptionType2 variable2) {  
    remedyBody2  
} ...  
...
```

Each *exceptionType_i* is the type of some exception, and each *variable_i* is a valid Java variable name.

The Java runtime environment begins performing a try-catch statement such as this by executing the block of statements, *guardedBody*. If no exceptions are generated during this execution, the flow of control continues with the first statement beyond the last line of the entire try-catch statement.

If, on the other hand, the block, *guardedBody*, generates an exception at some point, the execution of that block immediately terminates and execution jumps to the **catch** block whose *exceptionType* most closely matches the exception thrown (if any). The *variable* for this catch statement references the exception object itself, which can be used in the block of the matching **catch** statement. Once execution of that **catch** block completes, control flow continues with the first statement beyond the entire try-catch construct.

If an exception occurs during the execution of the block, *guardedBody*, that does not match any of the exception types declared in the catch statements, that exception is rethrown in the surrounding context.

There are several possible reactions when an exception is caught. One possibility is to print out an error message and terminate the program. There are also some interesting cases in which the best way to handle an exception is to quietly catch and ignore it (this can be done by having an empty body as a **catch** block). Another legitimate way of handling exceptions is to create and throw another exception, possibly one that specifies the exceptional condition more precisely.

We note briefly that try-catch statements in Java support a few advanced techniques that we will not use in this book. There can be an optional **finally** clause with a body that will be executed whether or not an exception happens in the original guarded body; this can be useful, for example, to close a file before proceeding onward. Java SE 7 introduced a new syntax known as a “try with resource” that provides even more advanced cleanup techniques for resources such as open files that must be properly cleaned up. Also as of Java SE 7, each catch statement can designate multiple exception types that it handles; previously, a separate clause would be needed for each one, even if the same remedy were applied in each case.

```
1 public static void main(String[] args) {  
2     int n = DEFAULT;  
3     try {  
4         n = Integer.parseInt(args[0]);  
5         if (n <= 0) {  
6             System.out.println("n must be positive. Using default.");  
7             n = DEFAULT;  
8         }  
9     } catch (ArrayIndexOutOfBoundsException e) {  
10        System.out.println("No argument specified for n. Using default.");  
11    } catch (NumberFormatException e) {  
12        System.out.println("Invalid integer argument. Using default.");  
13    }  
14 }
```

Code Fragment 2.13: A demonstration of catching an exception.

As a tangible example of a try-catch statement, we consider the simple application presented in Code Fragment 2.13. This main method attempts to interpret the first command-line argument as a positive integer. (Command-line arguments were introduced on page 16.)

The statement at risk of throwing an exception, at line 4, is the command `n = Integer.parseInt(args[0])`. That command may fail for one of two reasons. First, the attempt to access `args[0]` will fail if the user did not specify any arguments, and thus, the array `args` is empty. An `ArrayIndexOutOfBoundsException` will be thrown in that case (and caught by us at line 9). The second potential exception is when calling the `Integer.parseInt` method. That command succeeds so long as the parameter is a string that is a legitimate integer representation, such as "2013". Of course, since a command-line argument can be any string, the user might provide an invalid integer representation, in which case the `parseInt` method throws a `NumberFormatException` (caught by us at line 11).

A final condition we wish to enforce is that the integer specified by the user is positive. To test this property, we rely on a traditional conditional statement (lines 5–8). However, notice that we have placed that conditional statement within the primary body of the try-catch statement. That conditional statement will only be evaluated if the command at line 4 succeeded without exception; had an exception occurred at line 4, the primary try block is terminated, and control proceeds directly to the exception handling for the appropriate catch statement.

As an aside, if we had been willing to use the same error message for the two exceptional cases, we can use a single catch clause with the following syntax:

```
    } catch (ArrayIndexOutOfBoundsException | NumberFormatException e) {  
        System.out.println("Using default value for n.");  
    }
```

2.4.2 Throwing Exceptions

Exceptions originate when a piece of Java code finds some sort of problem during execution and **throws** an exception object. This is done by using the **throw** keyword followed by an instance of the exception type to be thrown. It is often convenient to instantiate an exception object at the time the exception has to be thrown. Thus, a **throw** statement is typically written as follows:

```
throw new exceptionType(parameters);
```

where *exceptionType* is the type of the exception and the parameters are sent to that type's constructor; most exception types offer a version of a constructor that accepts an error message string as a parameter.

As an example, the following method takes an integer parameter, which it expects to be positive. If a negative integer is sent, an `IllegalArgumentException` is thrown.

```
public void ensurePositive(int n) {  
    if (n < 0)  
        throw new IllegalArgumentException("That's not positive!");  
    // ...  
}
```

The execution of a **throw** statement immediately terminates the body of a method.

The Throws Clause

When a method is declared, it is possible to explicitly declare, as part of its signature, the possibility that a particular exception type may be thrown during a call to that method. It does not matter whether the exception is directly from a **throw** statement in that method body, or propagated upward from a secondary method call made from within the body.

The syntax for declaring possible exceptions in a method signature relies on the keyword **throws** (not to be confused with an actual **throw** statement). For example, the `parseInt` method of the `Integer` class has the following formal signature:

```
public static int parseInt(String s) throws NumberFormatException;
```

The designation “**throws** `NumberFormatException`” warns users about the possibility of an exceptional case, so that they might be better prepared to handle an exception that may arise. If one of many exception types may possibly be thrown, all such types can be listed, separated with commas. Alternatively, it may be possible to list an appropriate superclass that encompasses all specific exceptions that may be thrown.

The use of a **throws** clause in a method signature does not take away the responsibility of properly documenting all possible exceptions through the use of the `@throws` tag within a javadoc comment (see Section 1.9.4). The type and reasons for any potential exceptions should always be properly declared in the documentation for a method.

In contrast, the use of the **throws** clause in a method signature is optional for many types of exceptions. For example, the documentation for the `nextInt()` method of the `Scanner` class makes clear that three different exception types may arise:

- An `IllegalStateException`, if the scanner has been closed
- A `NoSuchElementException`, if the scanner is active, but there is currently no token available for input
- An `InputMismatchException`, if the next available token does not represent an integer

However, no potential exceptions are formally declared within the method signature; they are only noted in the documentation.

To better understand the functional purpose of the **throws** declaration in a method signature, it is helpful to know more about the way Java organizes its hierarchy of exception types.

2.4.3 Java's Exception Hierarchy

Java defines a rich inheritance hierarchy of all objects that are deemed `Throwable`. We show a small portion of this hierarchy in Figure 2.7. The hierarchy is intentionally divided into two subclasses: `Error` and `Exception`. **Errors** are typically thrown only by the Java Virtual Machine and designate the most serious situations that are unlikely to be recoverable, such as when the virtual machine is asked to execute a corrupt class file, or when the system runs out of memory. In contrast, **exceptions** designate situations in which a running program might reasonably be able to recover, for example, when unable to open a data file.

Checked and Unchecked Exceptions

Java provides further refinement by declaring the `RuntimeException` class as an important subclass of `Exception`. All subtypes of `RuntimeException` in Java are officially treated as **unchecked exceptions**, and any exception type that is not part of the `RuntimeException` is a **checked exception**.

The intent of the design is that runtime exceptions occur entirely due to mistakes in programming logic, such as using a bad index with an array, or sending an inappropriate value as a parameter to a method. While such programming errors

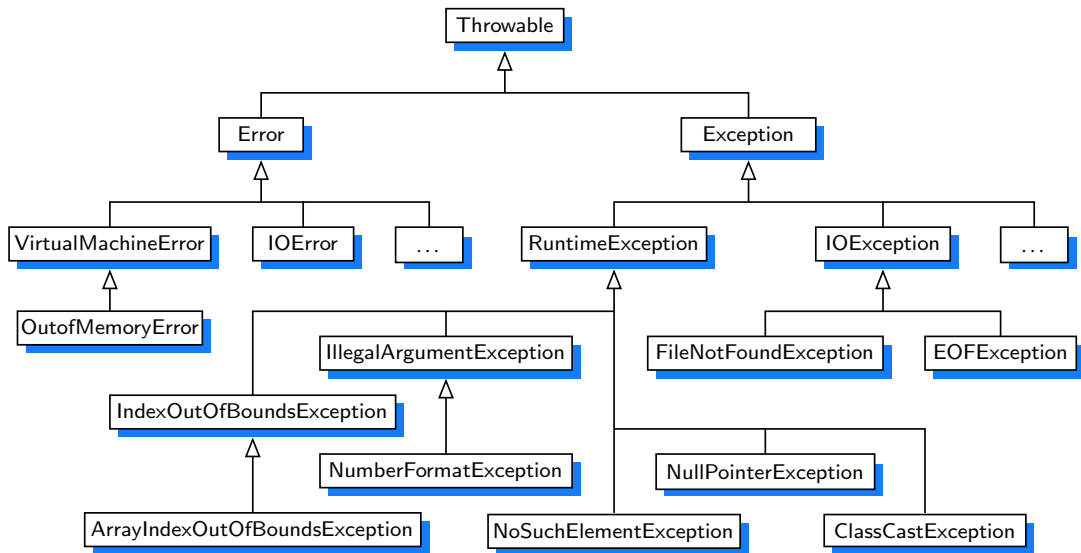


Figure 2.7: A small portion of Java's hierarchy of Throwable types.

will certainly occur as part of the software development process, they should presumably be resolved before software reaches production quality. Therefore, it is not in the interest of efficiency to explicitly check for each such mistake at runtime, and thus these are designated as “unchecked” exceptions.

In contrast, other exceptions occur because of conditions that cannot easily be detected until a program is executing, such as an unavailable file or a failed network connection. Those are typically designated as “checked” exceptions in Java (and thus, not a subtype of RuntimeException).

The designation between checked and unchecked exceptions plays a significant role in the syntax of the language. In particular, ***all checked exceptions that might propagate upward from a method must be explicitly declared in its signature.***

A consequence is that if one method calls a second method declaring checked exceptions, then the call to that second method must either be guarded within a try-catch statement, or else the calling method must itself declare the checked exceptions in its signature, since there is risk that such an exception might propagate upward from the calling method.

Defining New Exception Types

In this book, we will rely entirely on existing RuntimeException types to designate various requirements on the use of our data structures. However, some libraries define new classes of exceptions to describe more specific conditions. Specialized exceptions should inherit either from the Exception class (if checked), from the RuntimeException class (if unchecked), or from an existing Exception subtype that is more relevant.